

CSED312 Project 3

Design Report

팀 오우노

20230355 정지성

20230345 이성재

목차

1. Frame Table
2. Lazy Loading
3. Supplemental Page Table
4. Stack Growth
5. File Memory Mapping
6. Swap Table
7. On Process Termination

1. Frame Table

1.1. 관련 개념

먼저 프레임은 물리 메모리를 4kb 크기로 구분한 것으로, 가상 메모리의 페이지와 대응된다. 페이지와 마찬가지로 32비트 중 20비트는 프레임 넘버를 지칭하고, 나머지 12비트는 해당 프레임 내에서의 위치인 offset을 의미한다.

프레임 테이블은 physical memory의 단위인 프레임이 각각 어떤 프로세스(스레드)의 어떤 페이지에 할당되어 있는지를 관리하기 위한 자료구조이다. 이 외에도 어떤 프레임을 evict할지 결정하거나, 어떤 프레임이 free한 상태인지 쉽게 찾을 수 있도록 한다.

1.2. 현재 Pintos에서의 구현, 한계 및 개선점

현재 핀토스에는 프레임 테이블이 구현되어 있지 않다. 페이지를 관리하는 페이지 테이블은 `userprog/pagedir.c`에 정의되어 있지만 이는 각 프로세스가 갖고 있는 페이지 테이블로 단 하나 존재해야 하는 frame table은 구현이 되어 있지 않은 상태다. 현재 핀토스가 frame을 다루는 방식은 `threads/palloc.c`에서 user pool과 kernel pool로 할당가능한 frame들의 정보를 bitmap으로 저장하고 있는 것이 전부이다. 하지만 이렇게 하면 eviction을 진행해야 할 때 각 frame에 대해 아무런 정보도 가지고 있지 않아, 랜덤하게 eviction을 진행할 수밖에 없는 지경에 이른다. 따라서 더 나은 eviction algorithm을 위해 frame table을 정의한다.

1.3. 구현 계획

1.3.1. 자료 구조

Frame table은 그냥 linked list를 이용해 구현할 것이다.

Frame table의 각 entry에 대한 구성 요소에는 다음과 같은 정보가 있어야 한다. Frame의 free 여부는 palloc의 user pool bitmap에서 확인할 수 있으므로 넣지 않았다.

```
struct frame
{
    void *kaddr;           // frame의 커널 가상 주소
    void *uaddr;           // 매핑된 user virtual address
    struct thread *owner;   // 이 프레임을 쓰는 프로세스 (pagedir)

    bool pinned;           // eviction 금지 여부

    struct list_elem elem;  // 전역 frame_list용
};
```

1.3.2. 알고리즘

- `frame_alloc()`: user process이 부르는 함수이다. `palloc_get_page()`의 wrapper 함수라고 생각하면 된다. page 하나를 `palloc_get_page(PAL_USER)`로 할당해준 뒤, 정보들을 모아 frame list에 추가해주면 된다.
- `frame_dealloc()`: user process이 부르는 함수이다. `palloc_free_page()`를 부르고 frame을 frame list에서 제거해 주면 된다.
- `frame_evict()`: dealloc의 특수 케이스로, physical memory가 가득찬 경우 하나의 frame을 free해주는 함수이다. 기능이 복잡하므로 뒤에서 수도코드로 다시 정리하겠다.
- `frame_get_by_user(owner, address)`: frame list를 순회하며 owner와 address 정보를 가진 frame이 있으면 `struct frame`을 반환하고, 그렇지 않으면 `NULL`을 반환한다.

이제 `frame_evict()` 함수를 좀 더 잘 정의해보겠다. replacement algorithm이 출현하는 구간이며, 만약 evict되는 page가 dirty했었다면 file이나 swap에 수정사항을 반영하는 기능까지 넣어야 한다.

```
def frame_evict():
```

1. `clock_hand`가 가리키는 frame `f`를 가져온 후 `clock_hand`는 한 칸 전진
2. `f`가 pinned면 스킵
3. `f`의 accessed bit을 확인하고, accessed = true 였으면 false로 바꾸고 1로 돌아가기
4. `f`를 evict하기로 결정. `f`의 dirty bit이 켜져 있었으면, `f`과 관련된 process로 가 `f`의 S-page table entry의 상태에 맞게 file이나 swap disk를 업데이트.
5. `frame_dealloc(f)` 호출

2. Lazy Loading

2.1. 정의

Lazy loading은 프로그램을 실행할 때 관련된 코드와 데이터를 한꺼번에 메모리에 올리는 것이 아니라 필요할 때만 물리 메모리에 올리는 방법을 의미한다. 이러한 이유로 demand paging이라고 불리기도 한다. 이 방법을 구현할 때는 page fault도 다르게 처리할 필요가 있다. Page fault는 가상 메모리 주소에 접근했는데 대응되는 물리적 메모리 주소가 없을 때 발생한다. Lazy loading의 경우 이러한 상황이 일어났을 때 비로소 물리적 메모리 프레임을 할당하고 계속 프로그램을 실행해야 한다.

2.2. 현재 Pintos에서의 구현, 한계 및 개선점

현재 핀토스는 유저 프로그램을 실행할 때 모든 코드를 물리적 메모리에 할당한다. 이는 `userprog/process.c`의 `load_segment()` 함수에서 확인할 수 있다. 이 방식을 사용한다면 처음 load과정에서 이미 모든 필요한 데이터를 메모리에 할당했기 때문에, 이후 page fault가 일어나면 이는 illegal한 상황으로 간주하고 프로그램을 즉시 종료시킨다. 이는 `threads/exception.c`의 `page_fault()` 함수에서 확인할 수 있다.

하지만 이러한 구현 방식은 실제로 도달하지 않았을 수도 있는 코드들에게도 물리적 메모리를 미리 할당하기 때문에, 제한된 자원인 메모리를 낭비하는 구조이다. 정말 필요할 때만 메모리를 할당하도록 구현하면 메모리를 더 효율적으로 관리할 수 있다.

2.3. 구현 계획

Lazy loading을 high level에서 다음과 같이 구현할 수 있다.

- `load_segment()`함수에서 `palloc_get_page()`을 바로 하지 말고, 대신 supplemental page table의 엔트리를 추가시키는 것이다. 이 엔트리에는 읽고자 하는 파일의 struct file과 그 파일의 offset 등등을 저장하며, 이 page table entry는 executable을 읽기 위해 배정된 page이라는 정보를 남겨 두면 좋을 것이다.
- 이후 user process가 이렇게 initialize된 page를 읽고자 하면, page fault가 발생할 것이다. 이 때 user process가 읽고자 했던 address에 해당하는 supplemental page table을 구경해서 이 page가 executable을 읽기 위한 페이지라는 것을 알게 되고, 우리가 앞서 저장했던 struct file과 offset을 이용해 정보를 불러 오면 된다. 이에 대한 pseudocode는 3.2.2. Page fault logic에 정리했다.

3. Supplemental Page Table

3.1. 정의

Supplemental page table은 각 페이지에 대한 메타데이터를 저장함으로써 page table을 보완한다. Page table의 기본적인 목표는 주어진 가상 주소를 실제 물리 메모리 주소로 대응시키는 것이다. 운영체제는 추가적으로 프로그램 종료시 어떤 자원을 release할지, lazy loading으로 인한 page fault를 어떻게 처리할 것인지 등을 알고 있어야 한다. 이를 위해 도입된 시스템이 supplemental page table이다.

3.2. 구현 계획

3.2.1. Supplemental Page Table의 자료 구조

S-page table은 hash table로 구현할 것이다. user virtual address가 주어졌을 때 그에 해당하는 S-page table entry를 찾는 것이 가장 빠른 data structure이기 때문이다. 이에 따라 hash table의 key는 user page address로 두는 것이 자연스럽다.

S-page table에 존재하지 않는 page에 대한 address access는 illegal한 것으로 간주할 것이다. S-page table의 각 entry에 대한 구성 요소에는 다음과 같은 정보가 있어야 한다.

- User page address: 말그대로 해당 entry와 연결된 virtual page의 address이다. Hash table의 key로 사용되며, “page number”의 역할을 한다.
- Page의 status: **LOADED** / **LAZY** / **STACK** / **SWAP**. Page fault가 일어난 경우를 가정했을 때, **LAZY**의 경우 file system에서 정보를 읽어오고, **STACK**의 경우 0으로 채우며, **SWAP**의 경우 swap disk에서 정보를 읽어 온다. 여기서 **STACK** status는 현재 stack의 바로 아래 page 딱 한 개 존재하도록 디자인하겠다.
→ “possibility of frame allocation”의 정보는 이 status를 이용해 처리된다.
- 각 status에 대해 더 필요한 정보들
 1. **LOADED**: mapping된 frame table의 index
 2. **LAZY**: file pointer, offset from file (offset은 page 단위로 달라져야 한다)
 3. **SWAP**: entry index of swap table
- Hash table을 위한 **struct hash_elem**. Hash table의 key로는 user virtual address를 사용한다.

3.2.2. 알고리즘: Page Fault Logic

이제 Page fault으로 가능한 여러 가지 시나리오를 관찰해 보며 위의 S-page table이 어떻게 활용되는지 알아 보자.

def `page_fault_handler()`:

- Page fault이 발생하여 handler로 넘어감
- `fault_addr`이 속한 page address를 계산한 후, S-page table에 이 address가 존재하는지 검사 (없다면 illegal address)
- S-page table entry에서 page의 status 관찰. `frame_alloc()`로 allocation 후 **LAZY**, **STACK**, **SWAP**의 경우에 다른 연산 수행
 1. **LAZY**인 경우: file pointer와 file offset을 이용해서 데이터를 page에 load. Page의 상태를 **LOADED**으로 변경.
 2. **STACK**인 경우: 해당 page를 전부 0으로 초기화 후, 상태를 **LOADED**으로 변경. 또한, 지금 page의 바로 아래 page를 S-page table에 **STACK** status으로 추가.
 3. **SWAP**인 경우: swap table의 index를 활용해 swap slot을 찾은 후, 해당 데이터를 page에 load. Page의 상태를 **LOADED**으로 변경.

4. Stack Growth

4.1. 정의

Virtual memory의 구조를 보면, 유저 영역 맨 위에는 스택 영역이 있다. 스택은 로컬 변수 등이 저장되는 위치로 프로그램 실행에 필요한 데이터가 많아지면 아래로 자랄 수 있어야 한다. 이것이 일반적인 스택의 작동 방식이지만, 현재 핀토스에는 1 page size, 즉 4kb의 고정된 스택만을 할당해놓았다. 이를 필요에 따라 자랄 수 있게 만드는 것이 stack growth에서 해결해야 할 문제다.

4.2. 현재 구현 및 한계

프로세스를 만들 때 스택의 초기화가 필요하고 이는 `userprog/process.c`의 `setup_stack()` 함수에서 찾아볼 수 있다. 여기서는 `palloc_get_page()`를 호출해 하나의 페이지를 할당하고, 스택 포인터를 `PHYS_BASE`인 `0xc0000000`로 설정함으로써 1 페이지 크기의 스택을 초기화 한다. 하지만 이렇게 구현하면 프로세스가 더 많은 스택 영역을 필요로 할 경우 유동적으로 스택의 크기를 늘리지 못해 fault가 발생하게 된다. 스택의 크기를 무한정 늘릴 수 있는 것은 아니지만 요청이 있는 경우 필요한만큼 늘릴 수 있게 되면 물리적 메모리를 보다 효율적으로 활용할 수 있을 것이다.

4.3. 구현 계획

이는 앞의 3. Supplemental Page Table 구간에서 자세히 설명한 바 있으므로 넘어가겠다.

한 가지 언급할 점은, stack을 initialization할 때 초기의 첫 번째 stack allocation 이후, 바로 아래에 연결된 page 딱 한 개를 S-page table에 `STACK` status로 추가하면 괜찮은 구현이 될 것 같다.

5. File Memory Mapping

5.1. 정의

파일에 접근하기 위해서 기존에는 메모리에 파일 내용을 불러왔어야 했다. 버퍼를 이용해 내용을 불러오고, read/write 시스템 콜을 이용해 파일을 수정했다. File memory mapping은 파일을 마치 메모리에 접근하듯 할 수 있게 해주는 방식을 의미한다. File descriptor에 가상 메모리 주소를 대응시키고, 메모리 내용을 수정하듯이 파일의 내용을 수정하는 것이다. 이 방식의 좋은점은 lazy loading으로 구현이 된다는 것인데, 이렇게 하면 파일 중에서 실제로 필요한 부분만 메모리에 로딩해도 된다. 물론 페이지 단위로 접근은 해야 하지만, 파일의 크기가 크더라도 실제 필요한 부분이 적을 때 효과적인 방식이다.

5.2. 구현 계획

자료구조

- mapping table: 각 프로세스마다 따로 매핑 테이블을 갖고 있어야 한다. 따라서 thread 구조체에 `mmap_table` 이라는 이름의 리스트를 추가한다. 이 리스트는 `mmap_entry` 를 원소로 가지며 현재 계획 중인 `mmap_entry`의 구성요소는 대략 다음과 같다.

```
struct mmap_entry {  
    mapid_t mapid;  
    struct file *file;  
    void *start_addr;  
    size_t page_count;  
};
```

mapid는 마치 file descriptor 처럼 작동하며 1부터 시작한다.

알고리즘

핵심이 되는 두 시스템 콜을 구현해야한다.

- `mmap(int file_descriptor, void *mem_addr) -> mapid:`
 1. fd가 제대로 된 범위 내에 존재하는지 검사
 2. `mem_addr`이 올바른 주소를 갖고 있고 페이지 정렬이 되어 있는지 검사
 3. 필요한 페이지 수 계산
 4. 기존 페이지와 겹치는 주소가 포함되는지 확인
 5. `file_reopen()`으로 파일을 열고 새로운 S-page table entry 생성
 6. `mmap_table`에 새로운 `mmap_entry` 추가
 7. `mapid` 반환 및 1만큼 증가
- `munmap(mapid_t mapping):`
 1. `mmap_table` 순회하며 `mapping`을 `mapid`로 갖는 원소가 있는지 확인
 2. 없다면 아무것도 하지 않고 종료
 3. 1번 과정에서 찾은 `mmap_entry`의 `start_addr`부터 `page_count` 만큼 반복하며 페이지를 순회한다. 순회하며 해당 페이지가 S-page table에 포함되어 있는지 확인한다.

4. S-page table에 포함된 페이지들에 대해, 메모리에 로딩이 되어 있는지 확인하고 로딩되어 있지 않다면 5번 과정을 건너뛴다.
5. 메모리에 로딩되어 있는 각 페이지마다 dirty bit이 1인지 확인하고 1이라면 `file_write()`을 이용해 write back 해준다. 이후 페이지에 대응되는 프레임을 해제하고 페이지 테이블에서 제거한다.
6. S-page table에서도 4번 과정에서 처리한 페이지 원소들을 제거한다.
7. `mmap()`에서 파일을 다시 열어놓은 상황이기 때문에 `file_close()`로 파일을 닫아주고 종료한다.
8. `mmap_table`에서 현재 `mmap_entry`를 삭제한다.

6. Swap Table

6.1. 정의

컴퓨터에서 메모리는 매우 귀중한 자원이고, 여러 프로세스가 실행되면 메모리가 부족해질 수 있다. 메모리의 크기는 제한되어 있기 때문에, 디스크 드라이브를 메모리처럼 사용할 수 있게 해주는 swapping이 필요하다. 사용되지 않는 디스크의 일부를 마치 확장된 가상 메모리처럼 사용하게 해줌으로써 실제 물리 메모리에 올릴 수 있는 프로세스의 수 보다 많은 프로세스를 실행할 수 있게 한다. 이를 관리하기 위해 Swap table이라는 자료구조가 필요하다. 물리 메모리에 빈 자리가 없을 때, victim을 골라 swap 영역에 저장해놓은 후 메모리에는 free 상태로 표시하는데, 나중에 해당 영역에 접근할 경우 swap disk에서 내용을 가져오기 위해 어떤 메모리 주소에 해당하는 내용이 어떤 swap disk에 있는지 기록해둘 필요가 있다. 이를 swap table이 담당하며, swap disk에 존재하는 slot이 할당 되어 있는지 아닌지를 저장한다.

6.2. 현재 Pintos에서의 관련 코드 및 한계

Pintos 내부에서 Swap disk는 block device로의 abstraction이 되어 있다. `devices/block.c`에 정의된 함수를 이용해 swap disk를 다룰 수 있다. `block_get_handle(BLOCK_SWAP)`을 이용해 handle을 얻고, `block_read()`, `block_write()` 함수 등을 이용해 이 disk로의 read / write을 할 수 있다. 참고로 block device의 sector 크기는 512 bytes이고 swap slot의 크기는 4096 bytes이므로 한 swap slot 당 8개의 block sector을 할당하면 된다. 나중에 코드를 짤 때 유용한 정보일 것이다. 이 외에는 swap disk나 swap table 등에 대한 정의가 되어 있지 않다. 이렇게 되면 swapping을 진행하고자 할 때 swap disk의 어떤 데이터를 건드려도 되고 어떤 걸 건드리면 안 되는지에 대한 정보를 알아낼 수 없기 때문에, 별도의 swap table을 정의하여 사용하는 것이 좋아 보인다.

6.3. 구현 계획

bitmap data structure을 이용할 계획이다. Swap disk에 데이터를 저장하려 할 때 “swap disk의 이 자리가 사용중인가요?”만 판단하면 되기 때문에 이 data structure 하나로 충분해 보인다. Initialization은 우리가 사용할 `swap.dsk`에 들어가는 최대 swap slot의 개수를 길이로 bitmap을 만들고, 각 값을 0 (안 사용중)으로 초기화하면 될 것으로 예상된다. 이렇게 하면, bitmap의 `n`번째 비트는 우리의 `swap.dsk`에서 `n`번째 swap slot의 사용가능 여부에 대한 정보를 담고 있을 것이다.

이제 swap out과 swap in 기능을 pseudocode로 표현해 보면 다음과 같다.

```
def swap_out(frame_addr):
```

1. 비어 있는 슬롯 찾기; 빈 슬롯이 없다면 커널 패닉을 일으키고 종료
2. `block_write()`을 8번 호출해 총 8개의 섹터에 페이지 내용을 저장
3. swap out된 frame을 frame table에서 삭제시키기
4. swap out된 frame에 대해, 이 frame을 보고 있던 page의 status를 `SWAP`으로 바꾸어 주기
5. 사용한 슬롯에 대해 swap table의 비트를 1로 설정
6. 사용한 슬롯의 번호를 반환

```
def swap_in(slot_num):
```

1. `block_read()`를 8번 호출해 1 페이지에 해당하는 내용을 read
2. frame table에 추가시키기
3. 사용한 슬롯에 대해 swap table의 비트를 0으로 설정

7. On Process Termination

7.1. 정의 및 중요성

User process이 종료될 때, process과 관련되어 할당되었던 모든 memory를 deallocate하는 기능을 구현해야 한다. Deallocation을 제대로 수행하지 않으면 memory leak이 발생하여, 아무도 사용하지 않는데도 활용할 수 없는 memory가 늘어나서 os의 성능이 엄청나게 저하된다.

7.2. 구현 계획

사실 이 기능은 project 2에서도 구현하였다. Project 2에서는 process마다 `fd_table` 리스트, 그리고 open file마다 이 리스트 element에 해당하는 struct를 생성해주었다. 따라서, process가 종료되면 각각의 open file과 연관된 struct의 메모리를 반납해 주는 기능을 구현해야 했다. 이 기능은 `process_exit()` 함수에서 구현하였다.

비슷하게, project 3에서도 process 종료 전 할당한 메모리가 있었다면 `process_exit()` 함수에서 반납해 주면 될 것이다. 지금까지의 1~6장에서의 논의를 바탕으로, free되어야 하는 object들은 다음과 같다.

1. Process에 대응된 각각의 physical frame들
 - S-page table 중 **LOADED** status인 아이들에 대해, `palloc_free_page()`을 해 주고 frame table에서 제거해 주는 것으로 충분하다. (recall: 우리 구현에서는 frame table entry에 존재 == being used by some process이다.)
2. S-page table: **struct**를 정의했으므로 공간을 조금 차지한다. 각 entry를 free해주면 될 것이다.
3. Process에서 swap disk으로 swap out했던 swap slot들
 - 각각의 slot에 대한 swap table의 bitmap 값을 0으로 바꾸는 것으로 충분할 것이다.